UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

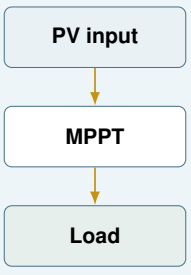
Department of Electrical, Electronic and Computer Engineering

MPPT Circuit Student Implementation Guide

Module ENR 420

Photovoltaic maximum power point tracking for safe laboratory implementation

A practical guide to the MPPT circuit, sensing interfaces, gate-driver control, STM32 pin mapping, firmware architecture, and safe laboratory bring-up used in Practicals 1 and 3.



solar energy management | power conversion | embedded control

Contents

1 How This Guide Should Be Used **1**
- 1.1 Assumed background . . . 1

2 System Overview **2**

3 Circuit Description **3**
- 3.1 Four-switch buck-boost converter . . . 3
- 3.2 Ratings and component values . . . 3

4 Safety and Operating Envelope **5**
- 4.1 Panel operating expectations . . . 5
- 4.2 Firmware fault limits . . . 5

5 Printed Circuit Board Configuration **6**

6 STM32 Interface and CubeMX Configuration **7**
- 6.1 CubeMX peripheral checklist . . . 7
- 6.2 Temperature Sensor Wiring . . . 8

7 Voltage and Current Sensing **9**
- 7.1 ADC conversion . . . 9
- 7.2 Voltage sensors . . . 9
- 7.3 Current sensors . . . 10
- 7.4 Power calculation . . . 10

8 Irradiance Sensor and RS485 Interface **11**
- 8.1 SEN0640 Wiring . . . 11
- 8.2 The Physical Layer: RS-485 Direction Control . . . 11
- 8.3 The Protocol Layer: Modbus-RTU . . . 11

9 Firmware Architecture **13**
- 9.1 Main control-loop timing . . . 13
- 9.2 State machine . . . 13
- 9.3 Guided MPPT and PSO pseudocode . . . 13
- 9.4 Fault handling . . . 14

10 Troubleshooting **15**

11 Reference Resources **16**

1 How This Guide Should Be Used

Welcome to your final-year power electronics project! If your previous experience was writing a basic PID controller on a microcontroller, this project is a fantastic step up. You are going to build the brain for a custom power electronics converter to extract the maximum possible power from solar panels.

This guide provides the system architecture, equations, hardware specifics, and printed circuit board (PCB) layouts you need to program the STM32 NUCLEO-F303K8 microcontroller. *However, it is not a finished solution.* The exact timing, control algorithms, digital filtering, and data parsing will rely on your engineering judgment to bridge the gaps.

Learning outcomes

After working through this guide you should be able to:

- explain how the four-switch buck-boost converter changes the solar-panel operating point;
- identify each PCB signal connected to the STM32 and configure the matching GPIO, ADC, UART, and timer peripherals;
- convert raw ADC counts into real-world voltage, current, power, and irradiance values;
- design a robust state machine that starts safely, checks measurements, limits duty cycles, and handles faults;
- communicate with digital sensors and your PC over serial protocols;
- verify the firmware in the laboratory before connecting the full solar-panel and battery system.

1.1 Assumed background

You are expected to know the basic operation of DC-DC converters, MOSFET switching, PWM, ADC sampling, UART communication, and C programming. The guide fills in the project-specific details: which signals exist on this PCB, how the sensor outputs relate to physical units, and how the STM32 firmware should be organised.

Safe laboratory sequence

Do not connect the solar panels, battery, or load during first firmware testing. First confirm that the STM32 boots, the gate-driver disable pins keep switching disabled, ADC readings are plausible, and PWM waveforms are correct on an oscilloscope. Only then proceed to current-limited converter testing.

2 System Overview

Solar panels are highly non-linear; their power output changes depending on how much current you draw from them. If you draw too little, voltage is high but current is near zero (no power). If you draw too much, voltage collapses to zero (no power).

Your goal is to find the "sweet spot" in the middle: the **Maximum Power Point (MPPT)**.

Because you cannot control the sun, you control the **duty cycle** of a DC-DC converter. By changing the duty cycle, you change the "electrical resistance" the solar panel sees, which shifts its operating voltage and current. The MPPT algorithm observes the resulting panel voltage and current, estimates $P_{\mathrm{PV}} = V_{\mathrm{PV}} I_{\mathrm{in}}$, and adjusts the converter command until the operating point is close to the peak.

Figure 1 shows the complete system with the PV input, measurement feedback, MPPT controller, PWM/gate-driver path, and four-switch buck-boost power stage. The MPPT PCB draws power from two series-connected Yingli YLM-J 3.0 PRO solar panels and delivers power to the output terminals.

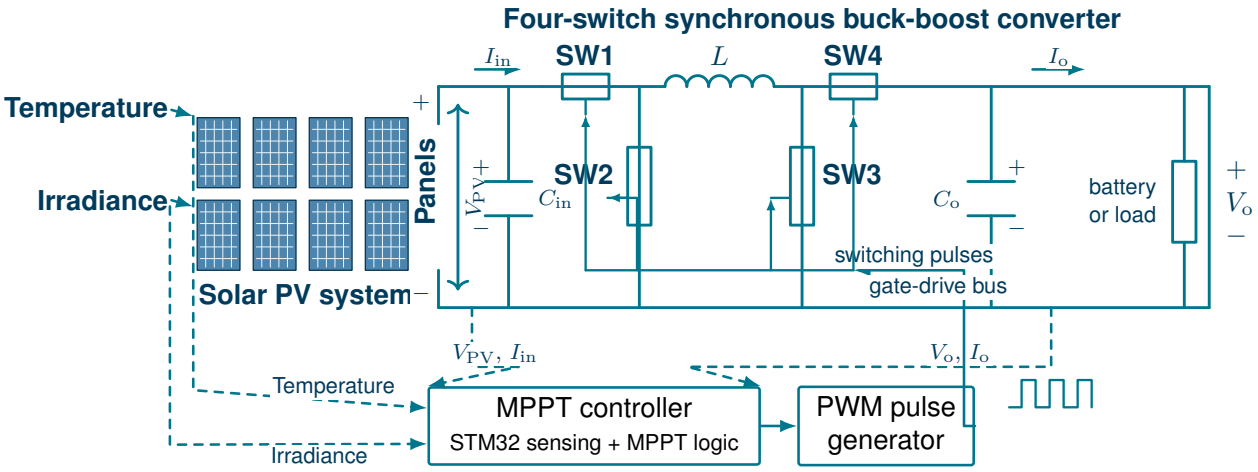


Figure 1: MPPT system diagram with four-switch buck-boost converter

Control objective

For Practical 3, the controller should search for the PV operating point that gives the highest measured input power, $P_{\mathrm{PV}} = V_{\mathrm{PV}} I_{\mathrm{in}}$, while keeping converter voltage, current, duty cycle, temperature, and communication conditions inside safe limits.

3 Circuit Description

3.1 Four-switch buck-boost converter

The DC-DC converter is a four-switch buck-boost (4SBB) converter. Because solar voltage fluctuates depending on irradiance and temperature (and your battery/load voltage varies), the 4SBB allows you to step the input voltage down or up without inverting the output polarity.

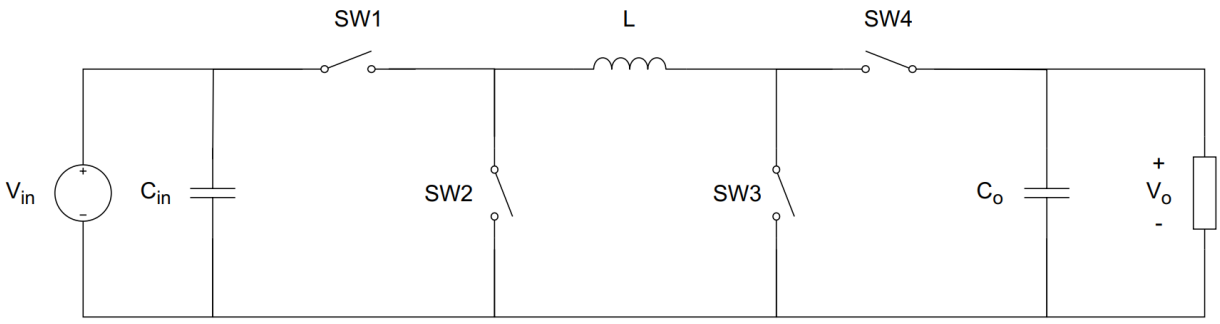


Figure 2: Four-switch buck-boost converter circuit diagram

SW1 and SW2 form the buck half bridge. SW3 and SW4 form the boost half bridge. The inductor transfers energy between the input side and the output side. The input and output capacitors reduce switching ripple and provide local energy storage.

Switch 1 and switch 2 operate with complementary duty cycles of D_1 and $1 - D_1$. Switch 3 and switch 4 operate with duty cycles of D_2 and $1 - D_2$. The ideal continuous-conduction voltage gain is

$$\frac{V_o}{V_{\text{in}}} = \frac{D_1}{1 - D_2}. \tag{1}$$

The useful operating cases are:

- **Buck mode**: set $D_2 = 0$. SW4 remains on, SW3 remains off, and the gain is approximately D_1.
- **Boost mode**: set $D_1 = 1$. SW1 remains on, SW2 remains off, and the gain is approximately $1/(1 - D_2)$.
- **Buck-boost mode**: set both sides to switch, for example $D_1 = D_2 = D$, giving a gain of approximately $D/(1 - D)$.

The firmware should not jump directly between extreme duty cycles. Use limited duty-cycle ranges, ramp commands gradually during startup, and stop switching immediately when a fault is detected.

3.2 Ratings and component values

The maximum ratings of the circuit are given in Table 1. These are hardware limits, not firmware targets. The controller must operate below these values and should include a safety margin.

Table 1: Circuit maximum ratings

Parameter	Value
Input voltage	$100\,\mathrm{V}$
Input current	$15\,\mathrm{A}$
Output voltage	$60\,\mathrm{V}$
Output current	$25\,\mathrm{A}$

The component values used for the inductor and capacitors are given in Table 2. The circuit is designed for a switching frequency of $250\,\mathrm{kHz}$.

Table 2: Component values

Parameter	Value
C_{in}	$25\,\mu\mathrm{F}$
L	$20\,\mu\mathrm{H}$
C_o	$15\,\mu\mathrm{F}$

At $250\,\mathrm{kHz}$, one PWM period is

$$T_s = \frac{1}{250\,000} = 4\,\mu\mathrm{s}. \tag{2}$$

The commanded duty cycles therefore change the relative on-time of MOSFETs within a $4\,\mu\mathrm{s}$ switching period.

4 Safety and Operating Envelope

The practical system can involve high DC voltage, high current, large capacitors, a battery, and outdoor PV panels. Firmware errors can therefore damage hardware. The first goal of the firmware is safe control. MPPT performance comes after safe startup, measurement sanity checks, and fault handling.

4.1 Panel operating expectations

Two series-connected YLM-J 3.0 PRO modules can produce an open-circuit voltage in the region of twice the single-module open-circuit voltage. A typical YLM-J 3.0 PRO 390–415 W module has V_{MPP} around 30–$31\,\mathrm{V}$, I_{MPP} around $13\,\mathrm{A}$, and V_{OC} around $37\,\mathrm{V}$ at standard test conditions. Two modules in series therefore place the expected maximum-power voltage near 60–$62\,\mathrm{V}$ and open-circuit voltage near $74\,\mathrm{V}$ before temperature corrections. This is below the PCB input-voltage rating, but it is still hazardous and must be treated as a high-energy DC source.

4.2 Firmware fault limits

Use conservative limits in early tests. The values in Table 3 are suggested starting limits for firmware development and should be confirmed by the demonstrator for the final practical setup.

Table 3: Suggested firmware fault limits for bring-up

Quantity	Initial action limit	Firmware action
Input voltage	Below hardware rating with margin	Disable PWM if the measured value exceeds the approved practical limit.
Input current	Current-limited during first tests	Disable PWM and log a fault if the input current rises unexpectedly.
Output voltage	Below battery/load rating	Disable PWM if output voltage is too high or inconsistent with the load.
Output current	Below practical limit	Disable PWM on overcurrent or negative current if not expected by the test.
Duty cycle	Keep away from 0% and 100%	Clamp commands and ramp slowly.
Sensor validity	ADC and UART values plausible	Do not run MPPT from invalid measurements.

5 Printed Circuit Board Configuration

A printed circuit board (PCB) has been designed and manufactured for the MPPT circuit. The circuit includes the converter, voltage sensors, current sensors, auxiliary power supplies, gate drivers, microcontroller headers, and irradiance-sensor interface. The unpopulated PCB is shown in Figures 3 and 4.

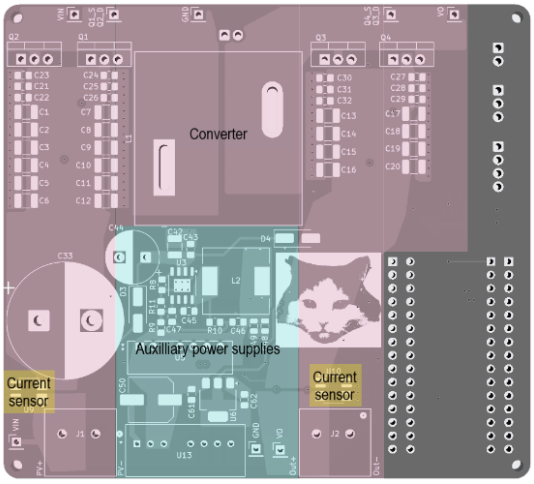


Figure 3: Unpopulated PCB top view

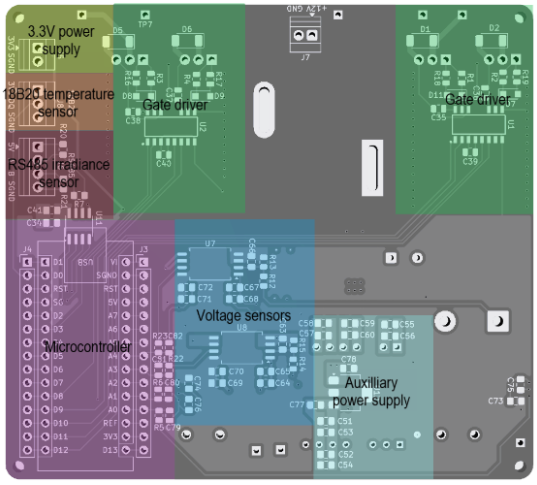


Figure 4: Unpopulated PCB bottom view

The PCB is pictured with components in Figures 5 and 6.

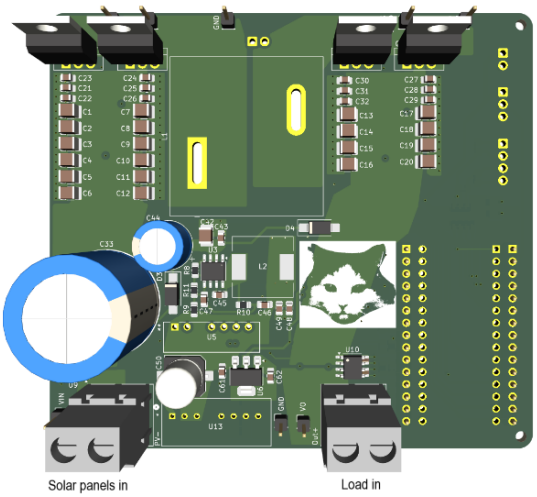


Figure 5: PCB top view with components

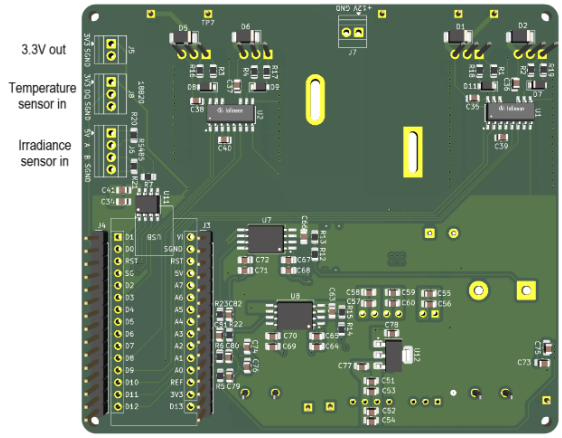


Figure 6: PCB bottom view with components

Connect the solar panels to the screw terminal labelled “Solar panels in”. Connect the controlled output load or battery connection to the output/load terminals according to the practical instructions.

Engineering Judgment: Do not rely strictly on printed PCB labels for high-power connections. Always verify the polarity of your panels and batteries with a multimeter before plugging them in to avoid catastrophic reverse-polarity damage.

6 STM32 Interface and CubeMX Configuration

The board is designed for an STM32 NUCLEO-F303K8 microcontroller. The NUCLEO board plugs directly into the PCB headers. Table 4 gives the practical signal map that you need when configuring the firmware in CubeMX.

Table 4: STM32 pin and PCB signal map

NUCLEO label	STM32 pin/function	PCB signal label	Purpose
D1	PA9 / `TIM1_CH2`	`BST_LOPWM`	PWM signal for the boost-side low switch (SW3).
D3	PB0 / `TIM1_CH2N`	`BST_HIPWM`	PWM signal for the boost-side high switch (SW4).
D9	PA8 / `TIM1_CH1`	`BCK_HIPWM`	PWM signal for the buck-side high switch (SW1).
D10	PA11 / `TIM1_CH1N`	`BCK_LOPWM`	PWM signal for the buck-side low switch (SW2).
D8	PF1 / GPIO output	`BST_DIS`	Active-high disable signal for the Infineon 2EDB8259Y boost-side gate driver.
D11	PB5 / GPIO output	`BCK_DIS`	Active-high disable signal for the buck-side gate driver.
D13	PB3 / GPIO output	`BST_STP`	Shoot-through or dead-time control signal for the boost side.
D12	PB4 / GPIO output	`BCK_STP`	Shoot-through or dead-time control signal for the buck side.
A0	PA0 / `ADC1_IN1`	`I_IN`	ADC feedback for input/PV current.
A1	PA1 / `ADC1_IN2`	`I_O`	ADC feedback for output current.
A3	PA4 / `ADC2_IN1`	`V_O`	ADC feedback for output voltage.
A4	PA5 / `ADC2_IN2`	`V_IN`	ADC feedback for input/PV voltage.
D0	PA10 / GPIO one-wire	`DQ`	Bidirectional data line for the DS18B20 temperature sensor.
D4	PB7 / UART RX	`RO`	Receive data from the ST4E1216IDT RS485 transceiver.
D5	PB6 / UART TX	`DI`	Transmit data to the ST4E1216IDT RS485 transceiver.
D6	PB1 / GPIO output	`RE`	RS485 receiver enable, active low.
D7	PF0 / GPIO output	`DE`	RS485 driver enable, active high.

PWM pin verification required

The official NUCLEO-F303K8 connector table maps D1 to PA9 and D3 to PB0. Therefore the boost-side D1/D3 channel labels must be checked against the actual PCB routing before energising the converter. Configure and probe the pins on an oscilloscope. If the observed high-side and low-side signals are reversed, correct the CubeMX mapping before continuing. Furthermore, **Dead-Time is critical**. If the top and bottom MOSFETs turn on simultaneously, a shoot-through short circuit will occur. You must configure the STM32's Advanced Timer to insert approximately 100 ns of dead-time between complimentary channel switching states.

6.1 CubeMX peripheral checklist

Configure the peripherals in a way that makes safe startup the default:

- **Clock**: set a stable system clock and confirm timer clock frequency before calculating PWM period.
- **TIM1**: use PWM generation with complementary outputs; enable preload; configure dead time; do not start outputs until all safe-state GPIOs have been set.
- **ADC1 and ADC2**: configure the four analog channels; use sufficient sample time for stable sensor readings; use DMA if continuous sampling is required.
- **GPIO**: set disable pins (`BCK_DIS`, `BST_DIS`) high immediately upon boot to keep gate drivers disabled safely.
- **USART**: configure the irradiance UART for the sensor's Modbus-RTU settings.
- **Debug**: keep SWD enabled so the firmware can be stopped and inspected during bring-up.

6.2 Temperature Sensor Wiring

The DS18B20 temperature sensor connects to the three-position screw terminal `J8`. With the PCB switched off, connect each lead to the matching silkscreen label in Table 5. Do not assume the terminal order from a photograph.

Table 5: DS18B20 connection to PCB terminal J8

J8 label	Sensor signal	Typical wire colour	Connection
`DQ`	Data	Yellow or white	Bidirectional 1-Wire data line to STM32 pin PA10 (D0).
`5V`	`VDD`	Red	Five-volt sensor supply.
`GND`	Ground	Black	Sensor ground and PCB reference.

Required 1-Wire pull-up resistor

The 1-Wire data line is open-drain and must be held high externally. Fit **one** $4.7\,\mathrm{k\Omega}$ **resistor between** `DQ` **and** `5V` at J8, preferably close to the connector. The resistor is connected across the two terminals; it is **not** placed in series with the data wire. One pull-up serves the complete bus, including when several DS18B20 sensors are connected in parallel.

Check the actual probe before power-up

Waterproof DS18B20 probe colours are not standardised across all suppliers. Confirm the supplied probe's labels or datasheet before applying power. Use the powered three-wire connection shown above, not a two-wire parasite-power connection.

7 Voltage and Current Sensing

The STM32 does not measure panel voltage or current directly. It measures sensor output voltages between approximately 0 and $3.3\,\mathrm{V}$. Firmware must convert ADC counts into engineering units before using them in MPPT or fault checks.

7.1 ADC conversion

For a 12-bit ADC with a reference voltage V_{REF}, the measured ADC input voltage is

$$V_{\mathrm{ADC}} = \frac{N}{4095} V_{\mathrm{REF}}, \tag{3}$$

where N is the raw ADC count. If the STM32 analog reference is $3.3\,\mathrm{V}$, then one count is approximately $0.806\,\mathrm{mV}$ at the ADC input.

Engineering Challenge: ADC readings are inherently noisy due to 250 kHz switching transients. If you feed raw, noisy readings into your MPPT algorithm, it will behave erratically. You must research, design, and implement a digital filter (such as an Exponential Moving Average) in software to smooth the data.

7.2 Voltage sensors

Two TI AMC0311S isolated amplifiers sense input and output voltages. The voltage divider feeds the isolated amplifier input, and the amplifier output is measured by the STM32 ADC. The circuit schematic is shown in Figure 7.

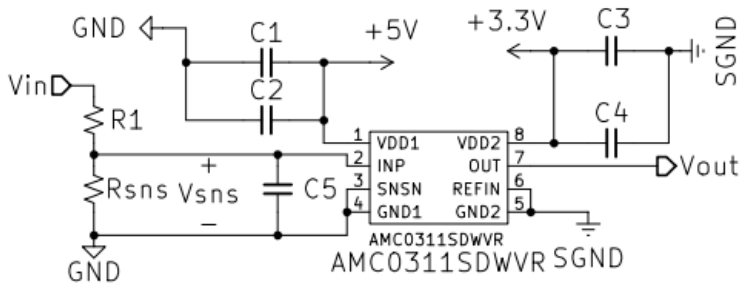


Figure 7: AMC0311S voltage sensor

The input voltage sensor is connected to `ADC2_IN2` on A4, and the output voltage sensor is connected to `ADC2_IN1` on A3.

Table 6: Voltage sensor parameter values

	Input voltage sensor	Output voltage sensor
R_1 (kΩ)	100	100
R_{sns} (kΩ)	2.2	3.6

Assuming unity gain from the AMC0311S signal path and the divider shown in Figure 7, the voltage

before the divider is estimated from

$$V_{\text{meas}} = V_{\text{ADC}} \left(\frac{R_1 + R_{\text{sns}}}{R_{\text{sns}}} \right). \tag{4}$$

Using the resistor values in Table 6, the approximate scale factors are:

$$V_{\text{in}} \approx V_{\text{ADC}}(46.45), \tag{5}$$
$$V_{\text{o}} \approx V_{\text{ADC}}(28.78). \tag{6}$$

7.3 Current sensors

Two Allegro CT416 magnetoresistive current sensors measure current. A CT416-HSN820DR unidirectional $20\,\text{A}$ sensor is used for the input current, and a CT416-HSN830MR bidirectional $\pm 30\,\text{A}$ sensor is used for the output current. The input current sensor output is connected to `ADC1_IN1` on A0. The output current sensor output is connected to `ADC1_IN2` on A1.

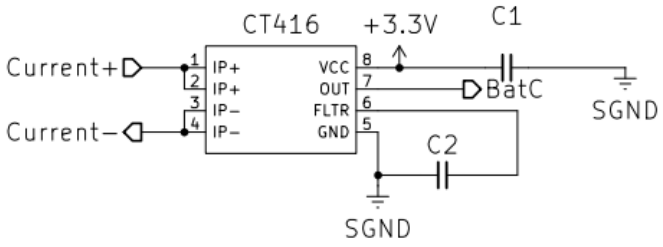


Figure 8: CT416 current sensor

For the CT416-HSN820DR input-current sensor, the nominal sensitivity is $100\,\text{mV/A}$. For a unidirectional sensor, the output is approximately

$$I_{\text{in}} \approx \frac{V_{\text{ADC}} - V_{\text{offset,in}}}{0.100}. \tag{7}$$

You will need to measure $V_{\text{offset,in}}$ at zero current in your initialization code before using the reading.

For the CT416-HSN830MR output-current sensor, the nominal sensitivity is $33.3\,\text{mV/A}$ and the zero-current output is near mid-supply. Estimate output current with

$$I_{\text{o}} \approx \frac{V_{\text{ADC}} - V_{\text{offset,o}}}{0.0333}. \tag{8}$$

7.4 Power calculation

The MPPT algorithm must use input-side PV power:

$$P_{\text{PV}} = V_{\text{in}} I_{\text{in}}. \tag{9}$$

Output voltage and output current are still important, but they are mainly used for battery/load monitoring, efficiency estimation, and fault detection.

8 Irradiance Sensor and RS485 Interface

A DFRobot SEN0640 photoelectric solar-radiation sensor is provided for irradiance sensing. It connects to screw terminal J7. The PCB uses an ST4E1216IDT RS485 transceiver to convert between the differential RS485 bus and the STM32 UART pins.

8.1 SEN0640 Wiring

With all supplies switched off, connect the four SEN0640 conductors by signal label as shown in Table 7.

Table 7: SEN0640 connection to PCB terminal J7

Wire colour	Sensor signal	J7 label	Connection
Brown	VCC	5V	Sensor supply input.
Yellow	485-A	A	RS485 differential line A.
Blue	485-B	B	RS485 differential line B.
Black	GND	SGND	Sensor power ground and RS485 signal reference.

Connection checks before power-up

Confirm approximately $5\,\mathrm{V}$ between J7 5V and SGND before inserting the sensor wires. Do not connect the brown wire to the separate $3.3\,\mathrm{V}$ output terminal. If communication fails, re-check A/B polarity without swapping any power connection.

The SEN0640 measures solar radiation over a 400–1100 nm wavelength range and reports values over **Modbus-RTU**. The default Modbus address is 0x01. The default UART settings are 4800 bit/s, 8 data bits, no parity, and 1 stop bit.

8.2 The Physical Layer: RS-485 Direction Control

RS-485 uses differential wires to reject noise. However, it is **half-duplex**: it can only transmit OR receive at one time, not both. The STM32 must explicitly control whether the transceiver is listening or transmitting using the Driver Enable (DE) and Receiver Enable (RE) pins.

- To **send** a request to the sensor: Set DE=1.
- To **listen** for the sensor's reply: Set DE=0 and RE=0.

The Gotcha: If you pull DE low *before* the STM32's UART has finished physically pushing the last bit out of the wire, your message will be corrupted. You must investigate the difference between the UART TXE (Transmit Data Register Empty) flag and the TC (Transmission Complete) flag in the STM32 HAL.

8.3 The Protocol Layer: Modbus-RTU

Modbus is a standardised way of formatting byte arrays. The sensor is the "Slave" (Address 0x01) and the STM32 is the "Master". To read the current solar radiation value, send a Modbus function 0x03 request for register 0x0000 with length 0x0001.

For the default address, the request frame is:

Listing 1: SEN0640 read-irradiance request bytes

```
01 03 00 00 00 01 84 0A
```

A response carrying $100\,\mathrm{W/m^2}$ is:

Listing 2: Example SEN0640 response bytes

```
01 03 02 00 64 9B AF
```

The Engineering Challenges:

1. **Endianness:** Modbus transmits data "Big-Endian" (most significant byte first). In the example above, `0x0064` equals 100. However, ARM Cortex-M processors (like your STM32) store data "Little-Endian". You will have to write code to swap the bytes when you receive the 16-bit irradiance value.
2. **CRC Verification:** Modbus requires a Cyclic Redundancy Check (CRC-16) at the end of every message to verify data integrity (e.g., `9B AF`). You will need to implement a standard Modbus CRC-16 algorithm in C to verify incoming messages and validate your own outgoing requests.

9 Firmware Architecture

When building power electronics, you **never** want to place all behaviour inside one large `while(1)` loop with hard-coded pin operations. You must implement the logic using a **State Machine**. Separate hardware drivers, measurement conversion, fault checks, and MPPT logic.

9.1 Main control-loop timing

You have to juggle tasks at different speeds:

- **250 kHz:** PWM generation (Hardware Timer).
- **Fast (1-10 kHz):** ADC acquisition (commonly using DMA) and Fault checking.
- **Medium (10-50 Hz):** MPPT update. Allow the converter and measurements to settle after changing the duty cycle.
- **Slow (1 Hz):** Irradiance polling and PC Data Logging.

Data Logging Warning: You need to send data to a computer over UART to save your results. Standard `printf()` over UART is extremely slow. If you call a blocking transmit function inside your main control loop, you will miss ADC measurements and potentially blow up the converter. Use non-blocking interrupts, DMA, or place your PC communication in a slow, strictly controlled timer block.

9.2 State machine

A suggested structure is shown in Table 8.

Table 8: Suggested firmware states

State	Purpose	Exit condition
`INIT`	Configure peripherals and force safe outputs.	Initialisation complete.
`IDLE`	PWM stopped, gate drivers disabled, measurements active.	User/test command and valid measurements.
`STARTUP`	Apply approved precharge/startup sequence and small duty command.	Voltages/currents plausible and stable.
`RUN`	Execute MPPT, update duty cycles, log data.	Fault, stop command, or test complete.
`FAULT`	Disable switching and record cause.	Manual reset or approved recovery condition.

9.3 Guided MPPT and PSO pseudocode

Particle swarm optimisation (PSO) can be used by treating each particle as a candidate converter command. The candidate may be a buck duty cycle, a boost duty cycle, or a mapped operating variable. Each particle stores its current position, velocity, personal best position, and personal best measured power.

This pseudocode provides a conceptual framework. You must determine the exact PSO coefficients and algorithm integration according to the practical instructions.

Listing 3: Guided PSO-style MPPT structure

```
initialise_particles_with_safe_duty_values();
global_best = first_valid_particle();

while (state == RUN) {
    read_and_filter_measurements();

    // Hardware limits must override the algorithm!
    check_fault_limits();
    if (fault_active) {
        disable_pwm_and_enter_fault();
        break;
    }

    particle = select_next_particle();

    // Never allow the optimiser to bypass safe duty clamps
    duty = clamp(particle.position, DUTY_MIN, DUTY_MAX);
    ramp_pwm_to(duty);
    wait_for_converter_to_settle();

    read_and_filter_measurements();
    p_pv = v_in * i_in;

    if (p_pv > particle.best_power) {
        particle.best_power = p_pv;
        particle.best_position = particle.position;
    }

    if (p_pv > global_best.power) {
        global_best.power = p_pv;
        global_best.position = particle.position;
    }

    update_particle_velocity_and_position(
        particle,
        particle.best_position,
        global_best.position);
}
```

9.4 Fault handling

When a fault occurs:

1. immediately command zero or safe PWM compare values;
2. assert `BCK_DIS` and `BST_DIS` to disable gate drivers;
3. stop MPPT updates;
4. record the fault source, latest measurements, and duty command;
5. require a deliberate reset or approved recovery action before re-enabling switching.

10 Troubleshooting

Table 9: Common faults and first checks

Symptom	Likely causes	First checks
No PWM on a pin	Timer not started, wrong alternate function, GPIO still configured as output/input	Check CubeMX pin mode, start calls, and oscilloscope ground.
Complementary pair overlaps	Dead time missing or wrong channel pairing	Recheck TIM1 complementary-output settings and D1/D3 mapping.
Gate driver does not switch	Disable pin high, supply missing, wrong PWM input	Measure `DIS`, driver supply, and input pin waveform.
ADC value stuck	Wrong ADC channel, pin not analog, DMA not started	Check CubeMX ADC rank/channel and raw counts.
Current nonzero at no load	Sensor offset not calibrated, noise, wrong sign convention	Record zero-current offset and apply filtering.
Irradiance timeout	RS485 direction wrong, A/B reversed, baud mismatch, wrong CRC	Check `DE`/`RE`, UART settings, and request bytes.
MPPT unstable	Update too fast, duty step too large, insufficient filtering, weak fault limits	Slow the MPPT loop, ramp duty, and log each candidate.
Output voltage rises too high	Load disconnected, duty too high, wrong mode transition	Disable PWM and inspect load, limits, and duty clamp.

11 Reference Resources

Use the following references when configuring firmware, implementing algorithms, or checking component behaviour:

- STM32CubeIDE download page.
- Introductory STM32CubeIDE and CubeMX setup video for getting started.
- NUCLEO-F303K8 product page.
- UM1956 STM32 Nucleo-32 boards user manual.
- STM32F303 documentation page.
- STM32F303x6/x8 datasheet.
- RM0316 STM32F303 reference manual for peripheral setup.
- Infineon 2EDB8259Y gate-driver page.
- TI AMC0311S isolated-amplifier page.
- Allegro CT416 current-sensor datasheet.
- Analog Devices DS18B20 temperature-sensor datasheet.
- ST4E1216 RS485 transceiver datasheet.
- Control Solutions Modbus protocol tutorial.
- DFRobot SEN0640 Modbus-RTU reference.
- Yingli YLM-J 3.0 PRO panel datasheet.